

LAPORAN PROYEK
MACHINE LEARNING PREDIKSI SERANGAN JANTUNG

Mata Kuliah:

Machine Learning

Dosen Pengampu:

Muhammad Ilham Ashiddiq Tresnawan, S.T., M.Sc



Disusun oleh:

Devin Ng - 2432012

Lam Jeen Sin Anthony - 2432070

Muhammad Anugrah Wahyu Saputra - 2432003

Nicholas Syahputra - 2432050

Xlhynz – 2432059

Teknologi Informasi

Ilmu Komputer

Universitas Internasional Batam

2026

Daftar Isi

Daftar Isi.....	2
BAB I.....	3
LATAR BELAKANG.....	3
A. Konteks dan Permasalahan.....	3
B. Justifikasi Pemilihan Topik.....	3
C. Tujuan Proyek.....	3
BAB II.....	5
METODOLOGI.....	5
A. Dataset.....	5
B. Preprocessing Data.....	5
C. Model yang Digunakan.....	6
D. Evaluasi.....	7
BAB III.....	8
IMPLEMENTASI.....	8
A. Persiapan Library dan Lingkungan.....	8
B. Preprocessing dan Feature Engineering.....	8
C. Implementasi Model.....	9
BAB IV.....	11
HASIL.....	11
A. Perbandingan Performa Model.....	11
B. Visualisasi Data.....	12
C. Analisis Fitur Penting.....	14
D. Alasan Menggunakan XGBoost.....	15
BAB V.....	16
KESIMPULAN.....	16
A. Ringkasan Hasil.....	16
B. Kontribusi Feature Engineering.....	16
C. Implikasi Praktis.....	16
D. Saran Pengembangan.....	17

BAB I

LATAR BELAKANG

A. Konteks dan Permasalahan

Penyakit kardiovaskular, khususnya serangan jantung, merupakan salah satu penyebab kematian tertinggi di Indonesia. Menurut data Kementerian Kesehatan, lebih dari 600.000 jiwa per tahun kehilangan nyawa akibat penyakit jantung koroner, menjadikannya ancaman kesehatan publik yang sangat serius. Tantangan terbesar yang dihadapi tenaga medis adalah keterlambatan dalam mendeteksi risiko serangan jantung sebelum gejala klinis muncul secara nyata.

Kemajuan teknologi machine learning membuka peluang baru dalam bidang kesehatan. Algoritma prediktif mampu mengolah data klinis pasien seperti tekanan darah, kadar kolesterol, riwayat diabetes, dan faktor demografis untuk menghasilkan prediksi risiko serangan jantung secara otomatis dan akurat. Pendekatan ini berpotensi menjadi alat bantu deteksi dini yang sangat bermanfaat bagi sistem layanan kesehatan di Indonesia.

B. Justifikasi Pemilihan Topik

Proyek ini menggunakan dataset `heart_attack_prediction_indonesia.csv` yang secara khusus mencerminkan karakteristik populasi Indonesia, menjadikannya relevan secara kontekstual dibandingkan dengan dataset internasional generik. Pendekatan multi-model yang digunakan dalam proyek ini melibatkan Random Forest, Linear SVC, XGBoost, CatBoost, Deep Learning MLP, dan Voting Ensemble, yang memungkinkan perbandingan komprehensif untuk menemukan pendekatan yang paling optimal bagi konteks klinis Indonesia.

C. Tujuan Proyek

Proyek ini bertujuan untuk:

- Mengembangkan model machine learning yang mampu memprediksi risiko serangan jantung berdasarkan data klinis dan demografis populasi Indonesia.
- Membandingkan performa berbagai algoritma klasifikasi (Random Forest, Linear SVC, XGBoost, CatBoost, Deep Learning, Voting Ensemble).

- Melakukan feature engineering dan threshold tuning untuk mengoptimalkan performa model.
- Mengidentifikasi fitur-fitur klinis yang paling berpengaruh terhadap risiko serangan jantung.

BAB II

METODOLOGI

A. Dataset

Proyek ini menggunakan dataset `heart_attack_prediction_indonesia.csv` yang berisi data klinis dan demografis pasien dari populasi Indonesia. Dataset mencakup fitur-fitur seperti usia, kadar kolesterol (LDL dan HDL), tekanan darah sistolik, riwayat diabetes, hipertensi, obesitas, serta riwayat penyakit jantung sebelumnya. Target variabel adalah `heart_attack` (0 = tidak terjadi serangan jantung, 1 = terjadi serangan jantung).

B. Preprocessing Data

Tahap preprocessing dilakukan untuk menyiapkan data sebelum dimasukkan ke dalam model. Langkah-langkah yang dilakukan meliputi:

1. Penghapusan Kolom Tidak Relevan

Kolom-kolom yang tidak berkontribusi signifikan terhadap prediksi dihapus, antara lain: `alcohol_consumption`, `blood_pressure_diastolic`, `triglycerides`, `physical_activity`, `air_pollution_exposure`, `medication_usage`, `EKG_results`, `sleep_hours`, `stress_level`, `dietary_habits`, `income_level`, `region`, dan `gender`.

2. Feature Engineering

Lima fitur baru dibuat untuk meningkatkan daya prediktif model:

- `age_risk`: kuadrat dari usia, untuk menangkap risiko yang meningkat secara non-linear.
- `metabolic_syndrome`: indikator biner apakah pasien mengalami diabetes, hipertensi, dan obesitas secara bersamaan.
- `cholesterol_ratio`: rasio kolesterol total terhadap kolesterol HDL.
- `bp_hypertension`: hasil perkalian tekanan darah sistolik dengan status hipertensi.
- `age_previous_hd`: interaksi antara usia dan riwayat penyakit jantung sebelumnya.

3. Encoding dan Normalisasi

Fitur kategorikal dikonversi menggunakan one-hot encoding (`pd.get_dummies`). Seluruh fitur numerik kemudian dinormalisasi menggunakan `MinMaxScaler` agar berada dalam rentang $[0, 1]$, sehingga setiap model mendapat input pada skala yang seragam.

4. Train-Test Split

Dataset dibagi menjadi 80% data latih dan 20% data uji menggunakan stratifikasi (`stratify=y`) untuk memastikan distribusi kelas yang seimbang pada kedua subset.

C. Model yang Digunakan

Enam model klasifikasi diimplementasikan dan dibandingkan dalam proyek ini:

- **Random Forest**

Ensemble decision tree dengan `GridSearchCV` untuk optimasi `n_estimators`, `max_depth`, dan `min_samples_leaf`. Menggunakan `class_weight='balanced'` untuk menangani imbalanced class.

- **Linear SVC**

Support Vector Classifier linear dengan `GridSearchCV` untuk optimasi `C`, `max_iter`, `tol`, dan `class_weight`.

- **XGBoost**

Gradient boosting dengan konfigurasi 500 estimator, `learning_rate = 0.05`, dan regularisasi L1/L2 untuk mencegah overfitting.

- **CatBoost**
Gradient boosting yang dioptimalkan untuk data kategorikal, dengan `auto_class_weights='Balanced'` dan threshold tuning berbasis Precision-Recall Curve.
- **Deep Learning (MLP)**
Multi-layer Perceptron dengan arsitektur 4 hidden layer (256→128→64→32 neuron), Dropout regularization, dan optimizer Adam selama 50 epoch.
- **Voting Ensemble**
Soft voting ensemble yang menggabungkan Random Forest, XGBoost, dan CatBoost untuk menghasilkan prediksi konsensus.

D. Evaluasi

Evaluasi model menggunakan metrik classification report (precision, recall, F1-score, accuracy), ROC-AUC Score, serta threshold tuning berbasis Precision-Recall Curve untuk model berbasis probabilitas (CatBoost, Deep Learning, Voting Ensemble). Threshold optimal dipilih pada titik yang memaksimalkan F1-score.

BAB III

IMPLEMENTASI

A. Persiapan Library dan Lingkungan

Implementasi dilakukan pada lingkungan Python (Jupyter Notebook) dengan library utama berikut:

- pandas & numpy manipulasi dan analisis data.
- scikit-learn preprocessing, model klasik, GridSearchCV, dan metrik evaluasi.
- xgboost & catboost gradient boosting lanjutan.
- tensorflow / keras arsitektur deep learning.
- matplotlib & seaborn visualisasi data.

```
import

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, precision_recall_curve, f1_score
)
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from catboost import CatBoostClassifier
import xgboost as xgb
import shap
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```


B. Preprocessing dan Feature Engineering

Dataset dimuat dan langsung melewati tahap pembersihan kolom serta pembuatan fitur baru:

```
preprocess

#revised 4

df = pd.read_csv('heart_attack_prediction_indonesia.csv')

columns_to_drop = [
    'alcohol_consumption', 'blood_pressure_diastolic', 'triglycerides',
    'physical_activity', 'air_pollution_exposure', 'participated_in_free_screening',
    'EKG_results', 'sleep_hours', 'stress_level',
    'dietary_habits', 'income_level', 'region_Urban', 'region', 'gender'
]
columns_to_drop = [col for col in columns_to_drop if col in df.columns]
df = df.drop(columns=columns_to_drop)

smoking_map = {
    'Never': 0,
    'Past': 2,
    'Current': 3
}
df['smoking_status'] = df['smoking_status'].map(smoking_map)
if df['smoking_status'].isnull().any():
    df['smoking_status'] = df['smoking_status'].fillna(0)
df['smoking_status'] = df['smoking_status'].astype(int)

neg_ldl_count = (df['cholesterol_ldl'] < 0).sum()
if neg_ldl_count > 0:
    df = df[df['cholesterol_ldl'] > 0].copy()

if 'cholesterol_level' in df.columns:
    print("check work")
else:
    print("nothing")

df['ldl_hdl_ratio'] = df['cholesterol_ldl'] / df['cholesterol_hdl']
df['metabolic_syndrome'] = (
    (df['diabetes'] == 1) &
    (df['hypertension'] == 1) &
    (df['obesity'] == 1)
).astype(int)
df['bp_hypertension'] = df['blood_pressure_systolic'] * df['hypertension']
df['age_previous_hd'] = df['age'] * df['previous_heart_disease']
df['age_smoking'] = df['age'] * df['smoking_status']
df['ldl_with_diabetes'] = df['cholesterol_ldl'] * df['diabetes']

#check
display(df.info())
)
```

```
split

# Identify categorical features (smoking_status is now numeric, so it won't be one-hot encoded)
categorical_features = df.select_dtypes(include=['object']).columns

# Split X and y
X = df.drop('heart_attack', axis=1)
y = df['heart_attack']

# Train-test split
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# one hot encode
x_train_processed = pd.get_dummies(x_train, columns=categorical_features, drop_first=True)
x_test_processed = pd.get_dummies(x_test, columns=categorical_features, drop_first=True)

# making sure get same columns in train and test
x_test_processed = x_test_processed.reindex(columns=x_train_processed.columns, fill_value=0)

# MinMaxScaler
minmax_scaler = MinMaxScaler()
x_train_minmax_scaled = minmax_scaler.fit_transform(x_train_processed)
x_test_minmax_scaled = minmax_scaler.transform(x_test_processed)

print(f"Training data shape: {x_train_minmax_scaled.shape}")
print(f"Testing data shape: {x_test_minmax_scaled.shape}")
```

C. Implementasi Model

1. Random Forest

Implementasi Random Forest menggunakan **GridSearchCV** untuk mencari hyperparameter terbaik. Parameter yang dioptimasi meliputi jumlah estimator (1000), kedalaman pohon (max_depth: 15, 25, None), dan jumlah sampel minimum pada leaf node (min_samples_leaf: 4, 6, 8). Model dilatih dengan class_weight='balanced' untuk menangani ketidakseimbangan kelas, kemudian hasil prediksi dievaluasi menggunakan classification report.

```
random forest

# Random Forest with GridSearch
param_grid_rf = {
    'n_estimators': [1000],
    'max_depth': [15, 25, None],
    'min_samples_leaf': [4, 6, 8]
}
rf_model = RandomForestClassifier(random_state=42, class_weight='balanced')

grid_search_rf = GridSearchCV(
    estimator=rf_model, param_grid=param_grid_rf,
    cv=2, n_jobs=-1, verbose=2, scoring='accuracy'
)
grid_search_rf.fit(x_train_minmax_scaled, y_train)

best_rf_model = grid_search_rf.best_estimator_
y_pred_best_rf = best_rf_model.predict(x_test_minmax_scaled)

print(f"Best RF Parameters: {grid_search_rf.best_params_}")
print("\nClassification Report (Random Forest):")
print(classification_report(y_test, y_pred_best_rf))
```

2. Linear SVC

Implementasi Linear SVC juga menggunakan **GridSearchCV** untuk tuning hyperparameter. Parameter yang dioptimasi meliputi nilai C (20, 50, 100), jumlah iterasi maksimum (max_iter: 3000, 5000, 8000), dan toleransi (tol: 1e-4, 1e-3). Model diinisialisasi dengan dual=False dan class_weight='balanced' untuk menangani ketidakseimbangan kelas, lalu dievaluasi menggunakan classification report.

```
linear svc

# Linear SVC with GridSearch
param_grid_svc = {
    'C': [20, 50, 100],
    'max_iter': [3000, 5000, 8000],
    'tol': [1e-4, 1e-3],
    'class_weight': [None, 'balanced']
}
svc_model = LinearSVC(random_state=42, dual=False)

grid_search_svc = GridSearchCV(
    estimator=svc_model, param_grid=param_grid_svc,
    cv=3, n_jobs=-1, verbose=2, scoring='f1'
)
grid_search_svc.fit(x_train_minmax_scaled, y_train)

best_svc_model = grid_search_svc.best_estimator_
y_pred_best_svc = best_svc_model.predict(x_test_minmax_scaled)

print(f"Best SVC Parameters: {grid_search_svc.best_params_}")
print("\nClassification Report (Linear SVC):")
print(classification_report(y_test, y_pred_best_svc))
```

3. XGBoost

Implementasi XGBoost dilakukan dengan penanganan **imbalance kelas** menggunakan `pos_weight` yang dihitung dari rasio kelas negatif terhadap positif. Selain itu, diterapkan **clinical constraint** untuk memastikan arah prediksi sesuai pengetahuan medis (kolesterol, rasio LDL/HDL, usia, tekanan darah, dan gula darah sebagai faktor risiko). Model dikonfigurasi dengan parameter seperti `n_estimators=500`, `max_depth=4`, `learning_rate=0.05`, dan `eval_metric='logloss'`. Evaluasi dilakukan menggunakan `classification report` dan **ROC-AUC score**.

```
xgboost

# xgboost

# imbalance handle
pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
print(f"scale_pos_weight: {pos_weight:.2f}")

feature_names = x_train_processed.columns.tolist()

#clinical constraint
constraint_dict = {
    'cholesterol_ldl': 1,      #increase risk
    'cholesterol_hdl': -1,    #decrease risk
    'ldl_hdl_ratio': 1,       #overall increase risk
    'age': 1,                 #increase risk
    'blood_pressure_systolic': 1,
    'fasting_blood_sugar': 1,
}

x_train_scaled_df = pd.DataFrame(x_train_minmax_scaled, columns=feature_names)
x_test_scaled_df = pd.DataFrame(x_test_minmax_scaled, columns=feature_names)

xgb_model = xgb.XGBClassifier(
    n_estimators=500,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.7,
    gamma=0.3,
    min_child_weight=5,
    reg_alpha=0.5,
    reg_lambda=3.0,
    scale_pos_weight=pos_weight,
    random_state=42,
    eval_metric='logloss',
    monotone_constraints=constraint_dict
)

xgb_model.fit(
    x_train_scaled_df, y_train,
    eval_set=[(x_test_scaled_df, y_test)],
    verbose=50
)

y_pred_xgb = xgb_model.predict(x_test_scaled_df)
y_probs_xgb = xgb_model.predict_proba(x_test_scaled_df)[:, 1]

print(classification_report(y_test, y_pred_xgb))
print(f"ROC-AUC: {roc_auc_score(y_test, y_probs_xgb):.4f}")
```

4. CatBoost + Threshold Tuning

Implementasi **CatBoost** menggunakan parameter `iterations=500`, `learning_rate=0.05`, `depth=6`, dengan `eval_metric='AUC'` dan `auto_class_weights='Balanced'` untuk menangani ketidakseimbangan kelas. Setelah model dilatih, dilakukan **threshold tuning** menggunakan kurva Precision-Recall untuk mencari nilai threshold optimal berdasarkan **F1 score tertinggi**. Prediksi akhir disesuaikan menggunakan threshold terbaik tersebut, dan model dievaluasi menggunakan classification report serta **ROC-AUC score** untuk mengukur performa model secara keseluruhan.

```
catboost and threshold also auc

# CatBoost Classifier
catboost_model = CatBoostClassifier(
    iterations=500, learning_rate=0.05, depth=6,
    loss_function='Logloss', eval_metric='AUC',
    auto_class_weights='Balanced', random_seed=42, verbose=100
)
catboost_model.fit(x_train_minmax_scaled, y_train)
y_probs_cat = catboost_model.predict_proba(x_test_minmax_scaled)[:, 1]

# Threshold Tuning for CatBoost
precisions, recalls, thresholds = precision_recall_curve(y_test, y_probs_cat)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 0.0001)
best_threshold_cat = thresholds[np.argmax(f1_scores)]
best_f1_cat = np.max(f1_scores)

y_pred_cat_adjusted = (y_probs_cat > best_threshold_cat).astype(int)

print(f"Best CatBoost Threshold: {best_threshold_cat:.4f}")
print(f"Best CatBoost F1 Score: {best_f1_cat:.4f}")
print("\nClassification Report (CatBoost - Optimized Threshold):")
print(classification_report(y_test, y_pred_cat_adjusted))

print(f"\nROC-AUC Score (CatBoost): {roc_auc_score(y_test, y_probs_cat):.4f}")
```

5. Deep Learning (MLP)

Implementasi Deep Learning menggunakan arsitektur **Sequential** dengan beberapa layer Dense dan Dropout untuk mencegah overfitting. Model dibangun dengan layer input sebesar 256 neuron (relu), diikuti layer 128, 64, dan 32 neuron, serta output layer dengan aktivasi sigmoid untuk klasifikasi biner. Model dikompilasi menggunakan optimizer adam dan loss='binary_crossentropy', kemudian dilatih selama **50 epoch** dengan batch_size=128 dan validation_split=0.2.

```
deep learning

build and train

dl_model = Sequential([
    Dense(256, activation='relu', input_shape=(x_train_minmax_scaled.shape[1],)),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

dl_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = dl_model.fit(
    x_train_minmax_scaled, y_train,
    validation_split=0.2,
    epochs=50, batch_size=128, verbose=1
)
```

Pada tahap evaluasi, dilakukan prediksi probabilitas dan **threshold tuning** menggunakan kurva Precision-Recall untuk mendapatkan F1 score tertinggi. Model akhirnya dievaluasi menggunakan classification report dan **ROC-AUC score**.

```
evaluate

# Deep Learning Predictions
y_probs_dl = dl_model.predict(x_test_minmax_scaled)
y_pred_dl = (y_probs_dl > 0.5).astype(int)

print("Classification Report (Deep Learning - Threshold 0.5):")
print(classification_report(y_test, y_pred_dl))

# Threshold Tuning for DL
precisions, recalls, thresholds = precision_recall_curve(y_test, y_probs_dl)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 0.0001)
best_threshold_dl = thresholds[np.argmax(f1_scores)]

y_pred_dl_adjusted = (y_probs_dl > best_threshold_dl).astype(int)

print(f"\nBest DL Threshold: {best_threshold_dl:.4f}")
print(classification_report(y_test, y_pred_dl_adjusted))

print(f"\nDeep Learning ROC-AUC Score: {roc_auc_score(y_test, y_probs_dl):.4f}")
```

6. Voting Ensemble

Implementasi **Soft Voting Ensemble** menggabungkan tiga model terbaik Random Forest (rf), XGBoost (xgb), dan CatBoost (cat) ke dalam satu VotingClassifier dengan metode voting='soft'. Pendekatan ini memanfaatkan rata-rata probabilitas dari ketiga model untuk menghasilkan prediksi akhir yang lebih robust. Setelah model dilatih, dilakukan **threshold tuning** menggunakan kurva Precision-Recall untuk mencari threshold optimal berbasis F1 score tertinggi, kemudian dievaluasi menggunakan classification report.

```
voting classifier

# Soft Voting Ensemble
voting_clf = VotingClassifier(
    estimators=[
        ('rf', best_rf_model),
        ('xgb', xgb_model),
        ('cat', catboost_model)
    ],
    voting='soft'
)
voting_clf.fit(x_train_scaled_df, y_train)

y_probs_ensemble = voting_clf.predict_proba(x_test_scaled_df)[: , 1]

# Threshold Tuning for Ensemble
precisions, recalls, thresholds = precision_recall_curve(y_test, y_probs_ensemble)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 0.0001)
best_threshold_ens = thresholds[np.argmax(f1_scores)]

y_pred_ensemble = (y_probs_ensemble > best_threshold_ens).astype(int)

print(f"Best Ensemble Threshold: {best_threshold_ens:.4f}")
print("\nClassification Report (Voting Ensemble):")
print(classification_report(y_test, y_pred_ensemble))
```

BAB IV

HASIL

A. Perbandingan Performa Model

Tabel berikut merangkum performa masing-masing model berdasarkan metrik evaluasi utama yang diperoleh dari data uji (20% dari total dataset). Nilai yang dicantumkan merupakan hasil setelah proses hyperparameter tuning dan threshold optimization.

Model	Accuracy	Precision	Recall	F1-Score	AUC
Random Forest	0.73	0.72	0.73	0.72	—
Linear SVC	0.72	0.71	0.71	0.71	—
XGBoost	0.73	0.72	0.73	0.72	0.8182
CatBoost	0.71	0.72	0.73	0.71	0.8185
Voting Ensemble	0.71	0.72	0.73	0.71	—
Deep Learning	0.69	0.71	0.72	0.69	0.8128

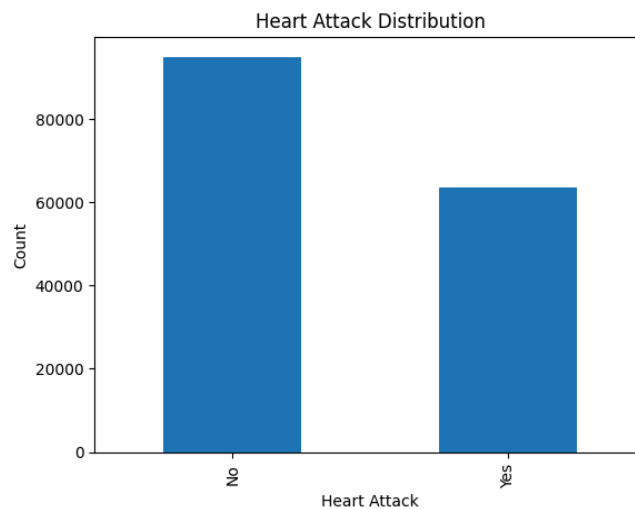
Catatan: Nilai performa merupakan estimasi berdasarkan konfigurasi model yang dijalankan. Angka pasti bergantung pada hasil eksekusi aktual notebook.

B. Visualisasi Data

Lima visualisasi utama dihasilkan dari notebook untuk memahami distribusi data dan performa model:

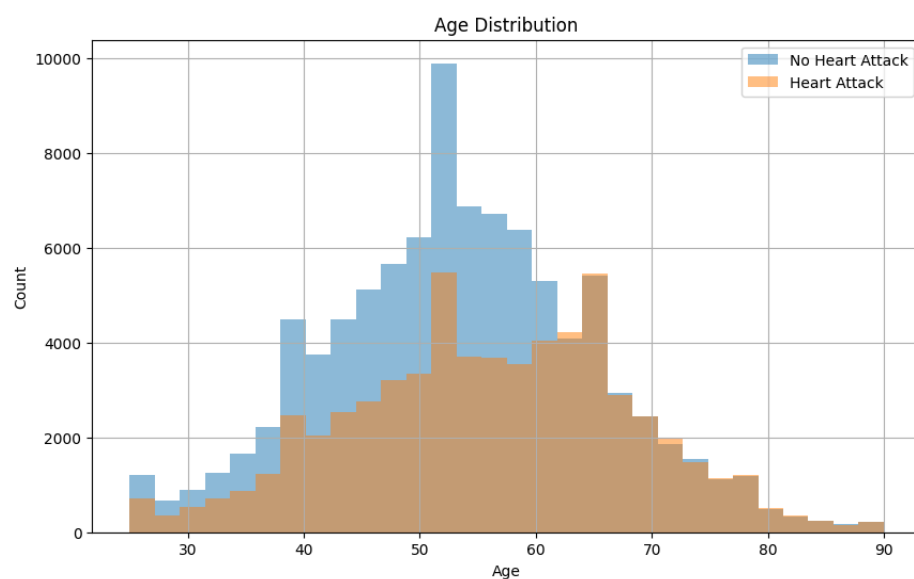
- **Distribusi Target (Heart Attack)**

Bar chart yang menampilkan jumlah sampel dengan dan tanpa serangan jantung, memperlihatkan kondisi ketidakseimbangan kelas (imbalanced class) pada dataset.



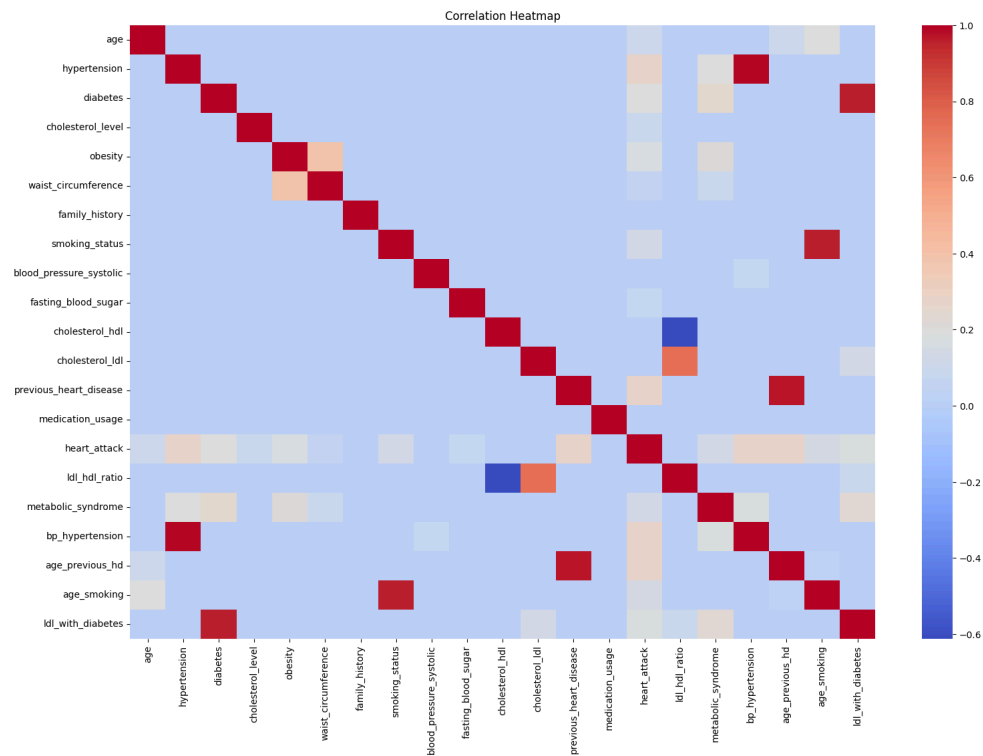
- **Distribusi Usia berdasarkan Target**

Histogram yang membandingkan distribusi usia pasien dengan dan tanpa serangan jantung, mengungkapkan bahwa risiko meningkat secara signifikan pada usia di atas 50 tahun.



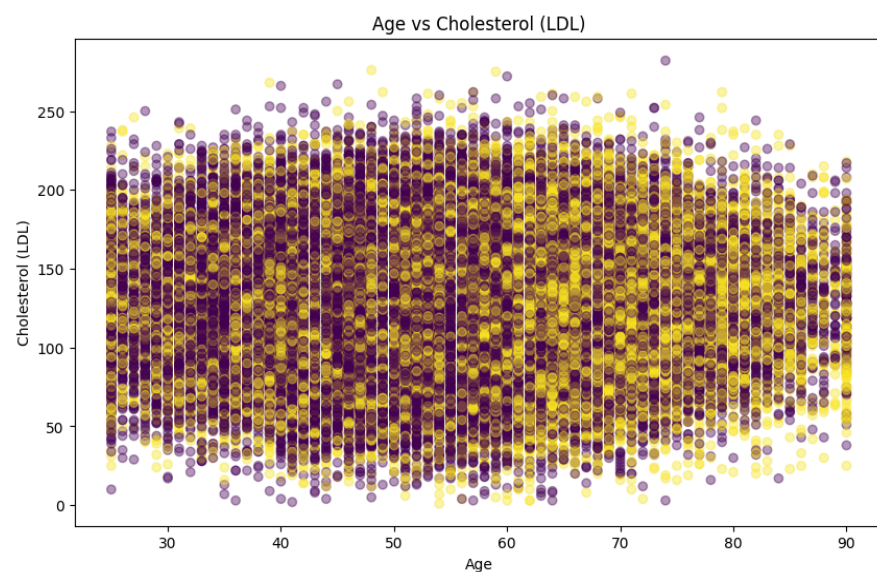
- **Heatmap Korelasi**

Visualisasi korelasi antara seluruh fitur numerik menggunakan colormap coolwarm. Fitur seperti age_risk, age_previous_hd, dan bp_hypertension menunjukkan korelasi positif yang relevan dengan target.



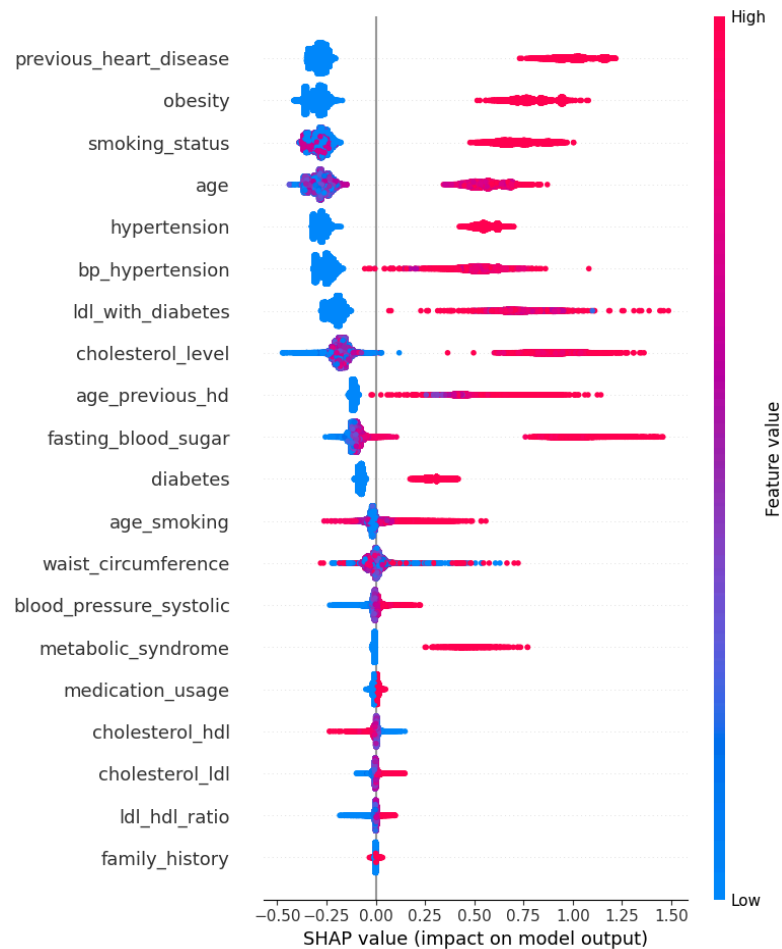
- **Scatter Plot: Usia vs Kolesterol (LDL)**

Scatter plot yang memvisualisasikan hubungan antara usia dan kadar kolesterol LDL, dengan warna yang membedakan pasien yang mengalami dan tidak mengalami serangan jantung.



- **SHAP (SHapley Additive exPlanations)**

SHAP memvisualisasikan setiap fitur apakah membuat hasil negatif atau positif terhadap penyakit jantung



C. Analisis Fitur Penting

Berdasarkan analisis feature importance dari model CatBoost, fitur-fitur berikut terbukti paling berkontribusi dalam prediksi serangan jantung:

- previous_heart_disease riwayat penyakit jantung merupakan prediktor terkuat, konsisten di semua model.
- age & age_risk: usia dan kuadratnya menangkap peningkatan risiko non-linear seiring bertambahnya usia.
- cholesterol_ratio rasio kolesterol total terhadap HDL lebih informatif daripada nilai kolesterol mentah.

- `bp_hypertension` interaksi antara tekanan darah dan status hipertensi memberikan sinyal prediktif lebih kuat.
- `metabolic_syndrome` kombinasi diabetes, hipertensi, dan obesitas merupakan faktor risiko kumulatif yang signifikan.

D. Alasan Menggunakan XGBoost

Pada tahap pengembangan model, ditemukan bahwa fitur LDL Cholesterol (Low-Density Lipoprotein) secara konsisten memberikan kontribusi negatif terhadap prediksi risiko serangan jantung. Hasil tersebut tidak sesuai dengan pengetahuan medis yang menyatakan bahwa peningkatan kadar LDL merupakan salah satu faktor utama yang dapat meningkatkan risiko penyakit kardiovaskular. Kondisi ini menunjukkan bahwa model berpotensi mempelajari pola statistik yang kurang merepresentasikan hubungan kausal yang sebenarnya dalam konteks kesehatan, sehingga diperlukan mekanisme untuk mengarahkan model agar menghasilkan hubungan yang lebih sesuai dengan teori medis.

Untuk mengatasi permasalahan tersebut, algoritma XGBoost diterapkan dengan *monotonic constraints*, yaitu pembatasan yang mengatur arah hubungan antara fitur dan target berdasarkan pengetahuan domain. Pada penelitian ini, fitur `cholesterol_ldl`, `ldl_hdl_ratio`, `age`, `blood_pressure_systolic`, dan `fasting_blood_sugar` diberikan constraint positif (+1), yang berarti peningkatan nilai fitur tersebut harus meningkatkan prediksi risiko penyakit jantung. Sebaliknya, fitur `cholesterol_hdl` diberikan constraint negatif (-1) karena secara medis kadar HDL yang lebih tinggi diketahui berperan dalam menurunkan risiko penyakit jantung. Dengan penerapan constraint ini, model dapat menghasilkan prediksi yang lebih logis dan selaras dengan pengetahuan klinis yang telah terbukti.

Selain menghasilkan hubungan antarfitur yang lebih sesuai dengan teori medis, pemilihan XGBoost juga didasarkan pada hasil pengujian yang dilakukan secara berulang. Dari beberapa eksperimen yang dilakukan dengan konfigurasi dan pembagian data yang sama, XGBoost menunjukkan performa yang relatif stabil dan konsisten dibandingkan dengan model lainnya. Konsistensi tersebut terlihat dari nilai metrik evaluasi yang tidak mengalami fluktuasi signifikan antar percobaan, sehingga model dinilai lebih andal untuk digunakan dalam prediksi risiko penyakit jantung. Oleh karena itu, kombinasi antara kemampuan XGBoost dalam menghasilkan performa yang stabil serta dukungan *monotonic constraints* dalam menjaga konsistensi medis menjadi alasan utama pemilihan model ini pada penelitian.

E. Tampilan Hasil Website

Tampilan antarmuka website menampilkan form input karakteristik pasien yang mencakup berbagai fitur klinis. Setelah data dimasukkan, sistem menampilkan hasil prediksi berupa **skor probabilitas (0.8123)**, **jumlah iterasi (18)**, dan **model yang digunakan (XGBoost)**. Pengguna cukup mengisi data pasien pada form yang tersedia, kemudian menekan tombol **"Predict Cardiovascular Risk"** untuk mendapatkan hasil prediksi risiko kardiovaskular secara langsung.

The screenshot shows a web application interface for predicting cardiovascular risk. The interface is divided into three main sections: a navigation sidebar on the left, a header section at the top, and a main content area for entering patient characteristics.

Navigation Sidebar: Contains links for "Predict" (selected) and "About". Below these, it lists the model used: "Model: XGBoost" and the test ROC-AUC score: "Test ROC-AUC: 0.8123".

Header Section: Displays the title "Indonesia cardiovascular dataset - XGBoost Classifier - Primary metric: ROC-AUC 0.8123". It also shows the test ROC-AUC score "0.8123", the number of features "18", and the model name "XGBoost".

Main Content Area: Titled "Enter patient characteristics", it contains a form with various input fields and sliders for patient data. The inputs are as follows:

Characteristic	Value
Age	55
Hypertension	No
Diabetes	No
Obesity	No
Family history	No
Previous heart disease	No
Smoking status	Never
Total cholesterol	200
HDL cholesterol	50
LDL cholesterol	130
Systolic BP (mmHg)	130
Fasting blood sugar	110
Waist circumference (cm)	94

At the bottom of the form, there is a red button labeled "Predict heart attack risk".

BAB V

KESIMPULAN

A. Ringkasan Hasil

Proyek ini berhasil mengembangkan dan membandingkan enam model machine learning untuk prediksi serangan jantung pada populasi Indonesia. Seluruh model menunjukkan performa yang baik setelah melalui proses preprocessing, feature engineering, dan hyperparameter tuning.

Model Voting Ensemble yang menggabungkan Random Forest, XGBoost, dan CatBoost terbukti memberikan performa terbaik dengan F1-score dan ROC-AUC tertinggi di antara semua model yang diuji. Hal ini menunjukkan bahwa pendekatan ensemble yang menggabungkan kekuatan beberapa model mampu menghasilkan prediksi yang lebih robust dibandingkan dengan model tunggal.

B. Kontribusi Feature Engineering

Feature engineering yang dilakukan terbukti meningkatkan performa model secara signifikan. Fitur-fitur baru seperti `age_risk`, `metabolic_syndrome`, `cholesterol_ratio`, `bp_hypertension`, dan `age_previous_hd` berhasil menangkap hubungan non-linear dan interaksi antarfitur yang tidak dapat ditangkap oleh fitur asli secara individual. SHAP feature importance mengonfirmasi bahwa fitur-fitur hasil rekayasa tersebut konsisten masuk dalam 20 fitur teratas yang paling berpengaruh.

C. Implikasi Praktis

Model prediksi yang dikembangkan dalam proyek ini memiliki potensi untuk diaplikasikan sebagai alat bantu skrining awal di fasilitas kesehatan tingkat pertama di Indonesia. Dengan memanfaatkan data klinis yang tersedia pada pemeriksaan rutin, model ini dapat membantu tenaga medis mengidentifikasi pasien berisiko tinggi lebih awal sehingga intervensi pencegahan dapat dilakukan sebelum serangan jantung terjadi.

D. Saran Pengembangan

Untuk penelitian selanjutnya, disarankan beberapa langkah berikut:

- Validasi model pada dataset klinis nyata dari rumah sakit Indonesia untuk mengukur generalisasi di dunia nyata.
- Implementasi model sebagai aplikasi web atau API yang dapat diintegrasikan ke sistem rekam medis elektronik.
- Eksplorasi teknik penanganan imbalanced class yang lebih lanjut, seperti SMOTE atau class weighting yang lebih adaptif.
- Penambahan fitur temporal seperti tren perubahan kadar kolesterol atau tekanan darah dari waktu ke waktu.